

```
syms o
f = @(x) (x*exp(x)-1)
```

```
f = function_handle with value:
    @(x)(x*exp(x)-1)
```

```
f_sym = f(o);
fd = derivataDeOrdinNPlusUnu(f,0)
```

```
fd =  $e^x + x e^x$ 
```

```
fd = matlabFunction(fd);
fdd = derivataDeOrdinNPlusUnu(f,1)
```

```
fdd =  $2 e^x + x e^x$ 
```

```
fdd = matlabFunction(fdd);

eroare = 1e-10;
maxIteratii = 200;
[z, ni] = Aproximare(f,fd,fdd,0,eroare, maxIteratii);

fprintf("Metoda lui Newton:");
```

```
Metoda lui Newton:
```

```
fprintf("f(x) = x*exp(x)-1");
```

```
f(x) = x*exp(x)-1
```

```
fprintf("Numărul de iterații efectuate: %d", ni);
```

```
Numărul de iterații efectuate: 4
```

```
fprintf("Valoarea este = %.10f", z);
```

```
Valoarea este = 0.5671432904
```

```
fprintf("Valoarea aproximata de Matlab prin fzero: %.10f", fzero(f, 0));
```

```
Valoarea aproximata de Matlab prin fzero: 0.5671432904
```

```
fprintf("Valoarea aproximata de Matlab prin solve: %.10f", solve(f_sym));
```

```
Valoarea aproximata de Matlab prin solve: 0.5671432904
```

Proceduri folosite:

1) pentru aproximare

```
function [z,ni]=Aproximare(f,fd,fdd,x0,eroarea,maxIteratii)
    if nargin < 6, maxIteratii=50; end
```

```

xp=x0(:);    %x precedent
for k=1:maxIteratii
    xc=xp-f(xp)/sqrt(fd(xp)^2-f(xp)*fdd(xp));
    if abs(xc-xp)<eroarea
        z=xc; %succes
        ni=k;
        return
    end
    xp=xc;
end
error('S-a depasit numarul maxim de iteratii');
end

```

2) pentru derivate

```

function [dfN] = derivataDeOrdinNPlusUnu(f, n)
    syms x;
    % Calculează derivata funcției f în raport cu x
    for i=1:n+1
        dfN = diff(f, x);
        f = dfN;
    end
end

```